

To develop a high-performing Machine Learning model for the **ExpansionRx-OpenADMET Blind Challenge**, we need a strategy that balances predictive power with the computational constraints of your environment.

Based on the **Kosmos report** and **leaderboard analysis**, the most effective approach for this dataset—given the limited compute—is a **hybrid feature engineering strategy** coupled with an **Ensemble of LightGBM Regressors**. This mimics the successful "Stacking Ensemble" approach (which combined GNNs and Tree models) by using rich molecular descriptors to approximate the structural insights usually gained from GNNs.

Modeling Strategy

1. Feature Engineering (The "Hybrid" Approach):

- **Structure:** Morgan Fingerprints (ECFP4, Radius 3, 2048 bits) to capture substructures.
- **Physicochemical:** RDKit 2D Descriptors (MW, LogP, TPSA, Ring Counts, etc.) to capture global properties.
- **Why:** Fingerprints act as the "visual" recognition (like the GNN in the Kosmos report), while descriptors provide the physical rules (like the physicochemical constraints).

2. Algorithm:

- **LightGBM (Gradient Boosting):** Chosen for its speed, low memory usage, and state-of-the-art performance on tabular chemical data.
- **Objective:** MAE (Mean Absolute Error) or Regression (L2). given the leaderboard metric is MA-RAE, minimizing MAE is mathematically aligned.

3. Data Preprocessing:

- **Log-Transformation:** Applied to skewed properties (Solubility, Clearance, Permeability) to normalize distributions, as suggested by the tutorial and standard practice.
- **Imputation:** LightGBM handles missing values natively, so we will not drop rows with partial data.

4. Ensembling:

- We will train **5 independent models** (different random seeds) for *each* of the 9 properties and average their predictions. This reduces variance and improves generalizability, crucial for a "blind" test set.

Python Solution

You can run the following code directly in your environment. It handles data loading, featurization, training, and generating the submission file.

Python

```
import pandas as pd

import numpy as np

import lightgbm as lgb

from rdkit import Chem

from rdkit.Chem import AllChem, Descriptors

from sklearn.model_selection import KFold

from tqdm import tqdm

import warnings

# Suppress warnings for cleaner output
warnings.filterwarnings('ignore')

# =====

# 1. Configuration & Helper Functions

# =====

TARGETS = [

    'LogD', 'KSOL', 'HLM CLint', 'MLM CLint',

    'Caco-2 Permeability Papp A>B', 'Caco-2 Permeability Efflux',
```

```

'MPPB', 'MBPB', 'MGMB'
]

# targets that benefit from Log-Transform (spanning orders of magnitude)
LOG_TRANSFORM_TARGETS = [
    'KSOL', 'HLM CLint', 'MLM CLint',
    'Caco-2 Permeability Papp A>B', 'Caco-2 Permeability Efflux'
]

def generate_features(smiles_series):
    """
    Generates a hybrid feature set: Morgan Fingerprints + RDKit Descriptors.
    """
    fp_list = []
    desc_list = []

    # Get list of descriptor functions
    descriptor_funcs = [x[1] for x in Descriptors.descList]
    descriptor_names = [x[0] for x in Descriptors.descList]

    print(f"Generating features for {len(smiles_series)} compounds...")

    for smi in tqdm(smiles_series):
        mol = Chem.MolFromSmiles(smi)

        if mol is None:
            # Handle invalid SMILES with a zero-vector placeholder

```

```
fp_list.append(np.zeros(2048))
desc_list.append(np.zeros(len(descriptor_names)))
continue
```

1. Morgan Fingerprint (ECFP4 equivalent)

```
fp = AllChem.GetMorganFingerprintAsBitVect(mol, radius=3, nBits=2048)
fp_list.append(np.array(fp))
```

2. RDKit Descriptors (Physicochemical props)

```
descs = []
for func in descriptor_funcs:
    try:
        val = func(mol)
        # Handle infinity or nan in descriptors
        if not np.isfinite(val):
            val = 0.0
    except:
        val = 0.0
    descs.append(val)
desc_list.append(descs)
```

Combine into a single matrix

```
X_fp = np.array(fp_list)
X_desc = np.array(desc_list)
```

Simple normalization for descriptors (LightGBM is robust, but this helps)

```

# We'll just pass raw, LightGBM handles scaling well.

# But concatenating them is key.
return np.hstack([X_fp, X_desc])

# =====

# 2. Load Data

# =====

print("Loading datasets...")

train_df = pd.read_csv("expansion_data_train.csv")
test_df = pd.read_csv("expansion_data_test_blinded.csv")

# =====

# 3. Feature Engineering

# =====

# Combine SMILES to generate features once (handles potential overlap/consistency)
train_smiles = train_df['SMILES'].tolist()
test_smiles = test_df['SMILES'].tolist()

X_train_full = generate_features(train_smiles)
X_test_full = generate_features(test_smiles)

print(f"Feature Matrix Shape: {X_train_full.shape}")

# =====

# 4. Training & Prediction (Multi-Task Ensemble)

# =====

```

```
submission_df = test_df[['Molecule Name']].copy()
```

```
for target in TARGETS:
```

```
    print(f"\nTraining models for Target: {target}")
```

```
    # Filter training data for this target (drop NaNs)
```

```
    mask = train_df[target].notna()
```

```
    y_train_raw = train_df.loc[mask, target].values
```

```
    X_train_curr = X_train_full[mask]
```

```
    # 4a. Log-Transform if applicable
```

```
    is_log = target in LOG_TRANSFORM_TARGETS
```

```
    if is_log:
```

```
        # Use log1p to handle zeros safely:  $y = \log(x + 1)$ 
```

```
        y_train = np.log1p(y_train_raw)
```

```
    else:
```

```
        y_train = y_train_raw
```

```
    # 4b. Ensemble Training (5 Seeds)
```

```
    n_seeds = 5
```

```
    test_preds_accum = np.zeros(X_test_full.shape[0])
```

```
    for i in range(n_seeds):
```

```
        seed = 42 + i
```

```
# LightGBM Parameters (Optimized for small-medium chemical datasets)
```

```
params = {  
    'objective': 'regression',  
    'metric': 'mae',  
    'boosting_type': 'gbdt',  
    'n_estimators': 1000,  
    'learning_rate': 0.05,  
    'num_leaves': 31,  
    'max_depth': -1,  
    'min_child_samples': 20,  
    'subsample': 0.8,  
    'colsample_bytree': 0.8,  
    'reg_alpha': 0.1,  
    'reg_lambda': 0.1,  
    'random_state': seed,  
    'n_jobs': -1,  
    'verbosity': -1  
}
```

```
# Simple split for early stopping
```

```
# In a real rigorous setting, we'd use scaffold split,
```

```
# but random split is sufficient for the "blind" ensemble component here.
```

```
from sklearn.model_selection import train_test_split
```

```
X_tr, X_val, y_tr, y_val = train_test_split(X_train_curr, y_train, test_size=0.1,  
random_state=seed)
```

```

model = lgb.LGBMRegressor(**params)

model.fit(
    X_tr, y_tr,
    eval_set=[(X_val, y_val)],
    callbacks=[lgb.early_stopping(stopping_rounds=50, verbose=False)]
)

# Predict on Test Set
preds = model.predict(X_test_full)
test_preds_accum += preds

# Average predictions
avg_preds = test_preds_accum / n_seeds

# 4c. Inverse Transform if applicable
if is_log:
    final_preds = np.expm1(avg_preds) # Inverse of log1p
    # Clip to ensure no negative values for physical properties
    final_preds = np.maximum(final_preds, 0)
else:
    final_preds = avg_preds

# 4d. Post-processing Constraints
# % Unbound targets should be clipped to [0, 100]
if 'Unbound' in target or target in ['MPPB', 'MBPB', 'MGMB']:
    final_preds = np.clip(final_preds, 0, 100)

```



```

submission_df[target] = final_preds

# =====

# 5. Save Submission

# =====

submission_filename = 'submission_expansion_admet.csv'
submission_df.to_csv(submission_filename, index=False)
print(f"\nProcessing complete! Predictions saved to {submission_filename}")
print("Head of submission:")
print(submission_df.head())

```

Why This Model Will Perform Well

- **Completeness:** By using both **fingerprints** (local structure) and **descriptors** (global properties like MW, TPSA), the model "sees" the molecule similarly to how a medicinal chemist does.
- **Robustness:** The **Ensemble (5 seeds)** averages out the noise inherent in small datasets, directly improving the Leaderboard score.
- **Data Handling:** Log-transforming the solubility (KSOL) and clearance (CLint) targets prevents the model from being biased by large outlier values, a common pitfall in ADMET modeling.
- **Efficiency:** LightGBM is extremely fast, respecting your VM's resource limits while delivering accuracy competitive with heavy Deep Learning models.

Next Steps

1. **Execute the code** above to generate your submission_expansion_admet.csv.
2. **Submit** this CSV to the leaderboard.
3. **Review the Kosmos Report (Trajectory r57):** If you have time for a second iteration, try implementing "Sample Weighting" where you give higher weight to training examples that are structurally similar to the test set (Tanimoto similarity), which Kosmos identified as a helpful strategy.